

The Wang Method Series

Notebook #9 (Version 1.0)

Understanding Edge Parity on Odd-Order Rubik's Cubes

A Transition-Based Method for 5×5, 7×7, 9×9, and Larger Odd-Order Cubes

Laung-Terng (LT) Wang

September 9, 2026

1. Introduction

A standard 3×3×3 Rubik's Cube is subject to orientation and permutation constraints that make certain apparent last-layer states impossible. In particular, a single flipped edge cannot occur through legal moves, and two edges cannot be exchanged by themselves while all other pieces remain fixed. These are the kinds of impossible reduced states commonly described as OLL parity and PLL parity on larger cubes.

Most higher-order cubes are solved using the 3×3 reduction method. For an odd-order cube such as the 5×5, 7×7, or 9×9, the solver first builds the six centers and pairs the edge pieces to form twelve complete composite edges.

After reduction is complete, the cube can be treated as an effective 3×3 and solved using only outer-layer moves with a familiar 3×3 method, such as the beginner method or beginner CFOP. The reduced state must satisfy the corresponding 3×3 orientation and permutation constraints. Thus a genuine single-composite-edge-flip or isolated two-composite-edge exchange cannot remain after complete reduction of an odd-order cube.

The parity studied in this notebook occurs earlier, during the pairing of the final composite edge, before reduction is complete. For this reason, we call it edge parity, or more specifically edge-pairing parity, rather than OLL parity or PLL parity.

The familiar long algorithm often called an “OLL parity algorithm” can still be used to repair this edge-pairing condition. However, the name of an algorithm does not determine the mathematical nature of the state it repairs.

This distinction provides the starting point for our study of 5×5, 7×7, 9×9, and general odd-order cubes. As the cube becomes larger, more wing pieces appear on each edge and the number of possible visible patterns grows. The goal of this notebook is to replace memorization of many cases with one systematic transition rule.

Consider a 7×7 edge on which Yellow (Y) is the reference color and Blue (B) is the opposite visible color. At first glance, we might imagine that, after all twelve edges have been paired, one edge could appear as YBBYYY or YYYYBBY.

These are not legal fully paired edge states. A 7×7 has two different wing orbits. In either proposed pattern, the two B wing positions belong to different wing orbits. A wing piece can be permuted among positions belonging to its own orbit, but it cannot become a wing piece of another orbit.

Therefore, the two B wings cannot be treated as two independently flippable pieces. The better question is: can two wing pieces from different orbits simply be exchanged with each other? The answer is no. The orbit structure prevents that exchange.

This gives a concrete reason why the proposed asymmetric patterns YBBYYYY and YYYYBBY cannot occur as fully paired last-edge states. It also motivates the key viewpoint used in this notebook: apparent wing orientation is constrained by wing permutation within each wing orbit.

Central Rule:

Count each visible-state transition ($0 \leftrightarrow 1$) from the outermost wing toward the midge. Each transition requires one appropriate edge-parity algorithm.

2. Midge, Wings, and Wing Orbits

Before using 0 and 1, we first need three simple terms.

Midge. On an odd-order cube, the piece at the exact middle of an edge is called the midge. There is one midge on each edge.

Wing. The other edge pieces on either side of the midge are called wings.

Wing orbit. Wings that are the same distance from the midge belong to the same wing orbit. An orbit contains all equivalent wing pieces around the cube. On one edge, we see two corresponding positions from each orbit, one on each side of the midge. A wing piece moves among positions belonging to its own orbit; it does not move into a different wing orbit.

5×5: $W_1 \mid M \mid W_1$ (1 wing orbit)

7×7: $W_1 \mid W_2 \mid M \mid W_2 \mid W_1$ (2 wing orbits)

9×9: $W_1 \mid W_2 \mid W_3 \mid M \mid W_3 \mid W_2 \mid W_1$ (3 wing orbits)

2.1 Using 0 and 1 to Describe Orientation

During edge pairing of the last edge, we use the orientation of the midge as a convenient visual reference. The midge can be any color; its actual color does not matter.

0 = same visible orientation state as the midge reference

1 = opposite visible orientation state

Therefore, 0 does not always mean Yellow and 1 does not always mean Blue. If the midge happens to be Yellow in a drawing, Yellow represents 0 only for that example. The binary symbols describe the visible state of a wing orbit; they do not mean that an individual wing can physically flip in place by itself.

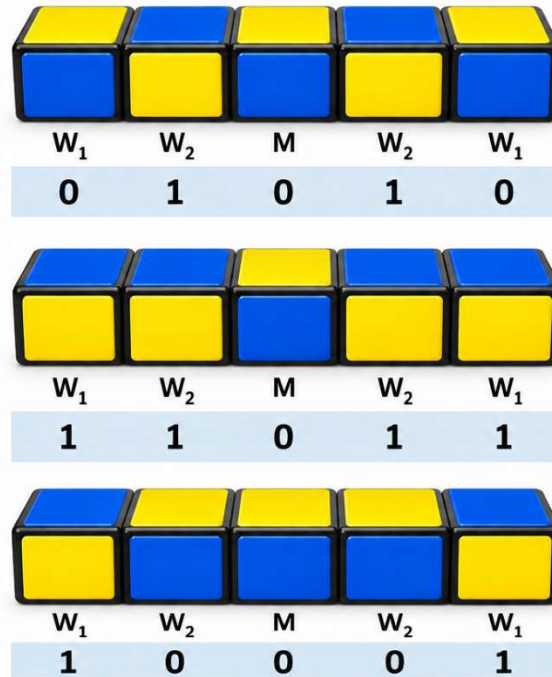
Figure 9.1 illustrates how the midge provides the reference for converting visible colors into binary states. Wings showing the same visible orientation as the midge are assigned 0, while those showing the opposite orientation are assigned 1.

2.2 Orientation as a Visible Effect of Wing Permutation

A crucial point is that a wing does not behave like an independently flippable 3×3 edge piece. Its apparent orientation at an edge location is tied to its permutation within its wing orbit. In a given edge location, such as UF, a wing from a particular orbit can occupy either of the two corresponding positions of that orbit. Moving between those positions changes the visible orientation relationship.

Thus, what looks visually like a wing-orientation problem is more fundamentally constrained by wing permutation. This is why asymmetric patterns such as YBBYYYY and YYYYBBY cannot simply be treated as additional last-edge parity cases. The binary 0/1 notation used in this notebook remains useful as a visual description, while the underlying physical mechanism is permutation within wing orbits.

Colors to Binary States



In these examples, Yellow (Y) = 0 and Blue (B) = 1, with the Yellow midge on the Up (Top) face as the reference.

Figure 9.1. Examples of converting visible 7×7 last-edge colors into binary orientation states.

3. The 5×5 Base Case

A 5×5 edge has one wing orbit surrounding the midge:

$$W_1 \mid M \mid W_1$$

In the last-edge parity patterns considered here, the two corresponding wing positions from the same orbit share one visible binary state. Therefore, one wing orbit gives only two possibilities: 0 or 1.

$$2^1 = 2$$

Table 9.1. Edge states for the 5×5 cube.

Full edge state	One side to midge	Transitions	Algorithms
000	00	0	0
101	10	1	1

The solved state 000 has no transition. In state 101, one side is 10. Moving toward the midge gives one orientation transition, $1 \rightarrow 0$, so one parity-algorithm application is required. This is the base case for the general deduction.

3.1 Algorithm 1 — 5×5 Edge-Parity Algorithm

For the 5×5 base case, the one-transition state 101 can be repaired with the following 18-move edge-parity algorithm:

$$Rw \ U2 \ x \ Rw \ U2 \ Rw \ U2 \ Rw' \ U2 \ Lw \ U2 \ 3Rw' \ U2 \ Rw \ U2 \ Rw' \ U2 \ Rw'$$

We call this Algorithm 1. It provides the 5×5 base algorithm before we introduce the two 7×7 layer-width variants.

4. The 7×7 Cube: Four Possible Patterns

A 7×7 edge has two wing orbits: an outer-wing orbit and an inner-wing orbit.

$$W_1 \mid W_2 \mid M \mid W_2 \mid W_1$$

For each orbit, the two corresponding wing positions are linked by the permutation structure of that wing orbit. In the last-edge parity patterns considered here, each orbit therefore contributes one independent visible binary state: 0 or 1. With two wing orbits, the 7×7 has

$$2^2 = 4$$

symmetric combinations, including the solved state: 00000, 10001, 01010, and 11011. An asymmetric string such as 00011 is not one of these cases because it assigns different visible states to corresponding positions of the same wing orbit. This is not merely a visual symmetry assumption; it reflects the underlying wing-orbit permutation constraint.

Table 9.2. Edge states for the 7×7 cube.

Full edge state	One side to midge	Transitions	Algorithms
00000	000	0	0
10001	100	1	1
01010	010	2	2
11011	110	1	1

4.1 The Two 7×7 Edge-Parity Algorithm Variants

For the 7×7 cube, there are two possible transition boundaries, so two layer-width versions of the edge-parity algorithm are needed. Continuing the numbering after the 5×5 base case, we call them Algorithm 2 and Algorithm 3.

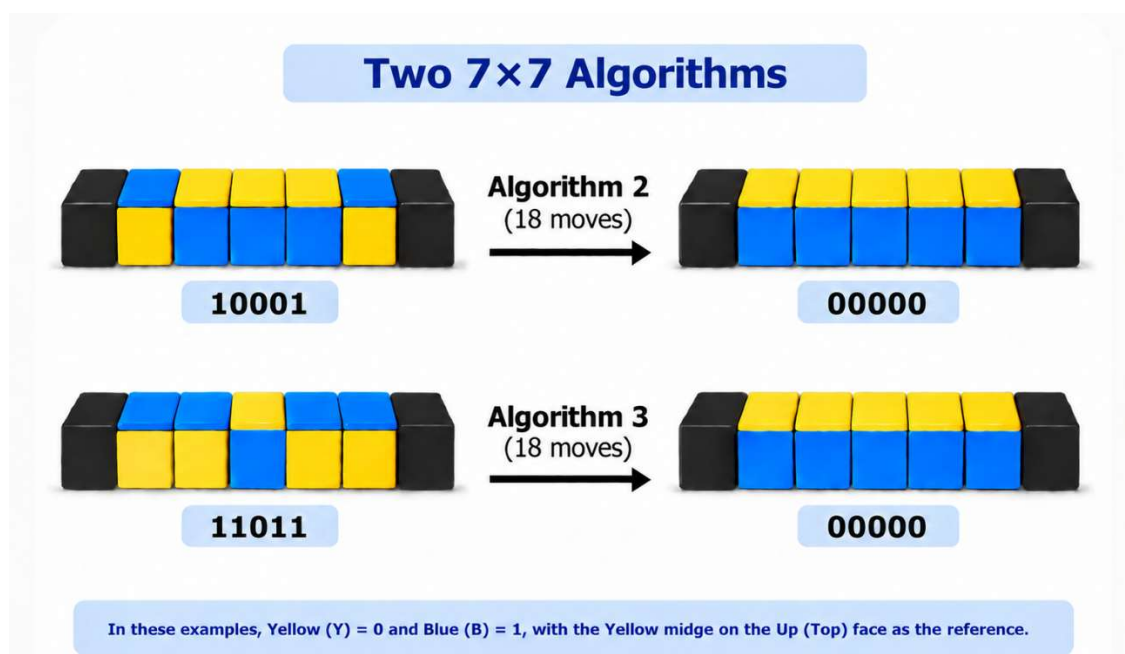


Figure 9.2. The two 18-move edge-parity algorithms needed for the two one-transition 7×7 cases.

Algorithm 2 — solves $10001 \rightarrow 00000$ (18 moves)

Rw U2 x Rw U2 Rw U2 Rw' U2 Lw U2 5Rw' U2 Rw U2 Rw' U2 Rw'

Algorithm 3 — solves $11011 \rightarrow 00000$ (18 moves)

3Rw U2 x 3Rw U2 3Rw U2 3Rw' U2 3Lw U2 4Rw' U2 3Rw U2 3Rw' U2 3Rw'

Figure 9.2 illustrates the two one-transition cases on the 7×7 cube. Algorithm 2 repairs the outer transition in 10001 , while Algorithm 3 repairs the inner transition in 11011 , bringing either pattern to the solved state 00000 .

4.2 The Two-Transition Case: 01010

The state 01010 is especially useful because one side together with the midge is 010 . It contains two orientation transitions:

$$0 \rightarrow 1 \rightarrow 0$$

Therefore, two algorithm applications are needed. For the 7×7 , Algorithm 2 is applied first, producing the intermediate state 11011 . Algorithm 3 then solves that state.

$$01010 \rightarrow 11011 \rightarrow 00000$$

Figure 9.3 shows how the two algorithms work together when both transition boundaries are present. Algorithm 2 first changes 01010 to 11011 by repairing the outer transition; Algorithm 3 then repairs the remaining inner transition, producing the solved state 00000 .

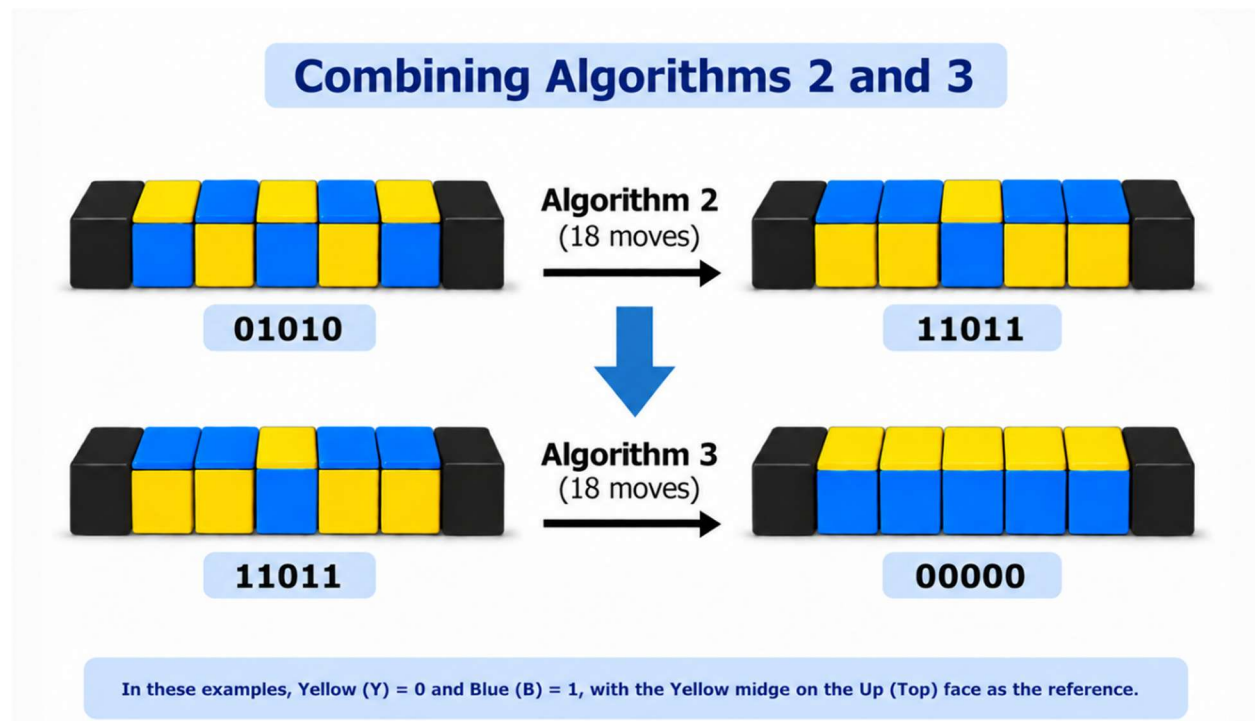


Figure 9.3. The 7×7 case 01010 requires two transitions and is solved by Algorithm 2 followed by Algorithm 3.

5. Extension to the 9×9 Cube

A 9×9 edge has three wing orbits surrounding the midge. Therefore, there are

$$2^3 = 8$$

symmetric binary states, including the solved state. There are also three possible transition boundaries, so a 9×9 needs three corresponding layer-width versions of the edge-parity algorithm. Their exact layer widths depend on which of the three transition boundaries is being repaired.

Table 9.3. Edge states for the 9×9 cube.

Full edge state	One side to midge	Transitions	Algorithm applications
0000000	0000	0	0
1000001	1000	1	1
0100010	0100	2	2
0010100	0010	2	2
1100011	1100	1	1
1010101	1010	3	3
0110110	0110	2	2
1110111	1110	1	1

The table shows why the number of 1s is not the answer. For example, 1110 has only one transition, while 1010 has three transitions. The required algorithms are determined by the transition boundaries, not simply by how many wing orbits are in state 1.

6. The Transition Rule

For any odd-order cube, write the edge as a symmetric binary sequence around the midge. Because corresponding positions from each wing orbit have the same state, only one side together with the midge needs to be inspected.

Number of required algorithm applications = number of adjacent visible-state transitions from the outermost wing to the midge.

Equivalently, the transitions may be counted from the midge outward; the number of transitions is the same in either direction.

A transition occurs whenever two neighboring visible states differ, $0 \leftrightarrow 1$. Each possible transition boundary has a corresponding layer-width version of the parity algorithm. Apply the appropriate version for each transition boundary until the last edge is paired.

7. Why the Rule Works for Every Odd-Order Cube

We do not need to memorize or experimentally list every case for 11×11, 13×13, and larger odd-order cubes. The rule follows by induction.

7.1 Base Case: 5×5

The 5×5 has one wing orbit. Its two states, 000 and 101, require zero and one algorithm application. Thus the transition rule holds for the base case.

7.2 Inductive Step

Increasing an odd-order cube from $N \times N$ to $(N+2) \times (N+2)$ adds exactly one new outer wing orbit. This adds one new binary state to the one-sided representation and therefore at most one additional transition.

$$N \times N \rightarrow (N + 2) \times (N + 2)$$

Assume the transition rule is correct for an odd $N \times N$ cube. When the new outer wing orbit is added, all of the old inner states and their transitions stay the same. There is only one new comparison: the new outer state versus the state that was previously outermost.

If the two states are the same, no new transition is created. If they are different, exactly one new transition is created. Therefore, moving from $N \times N$ to $(N+2) \times (N+2)$ adds either zero or one required algorithm application. This is the inductive step.

Since the rule is true for 5×5 and remains true each time one new wing orbit is added, it applies to all odd-order cubes of size 5×5 and larger.

7.3 General Mathematical Form

For an odd $N \times N$ cube, the number of wing orbits is

$$m = \frac{N - 3}{2}$$

Each wing orbit has two possible orientation states, 0 or 1. Therefore, the number of symmetric binary edge states is

$$2^m = 2^{\frac{N - 3}{2}}$$

Let $b_1, b_2, \dots, b_m$ be the one-sided wing-orbit states from the outermost wing toward the midge, and let the midge be $b_{m+1} = 0$. Then the number A of required algorithm applications is the number of adjacent state changes:

$$A = \sum_{i=1}^m |b_i - b_{i+1}|$$

The maximum occurs when the states alternate at every possible boundary:

$$A_{\max} = \frac{N - 3}{2}$$

8. A Beginner's Solving Procedure

1. At the last edge during edge pairing, use the midge orientation as the visual reference state 0.
2. Write each wing orbit as 0 if it shows the same visible orientation state as the midge reference, or 1 if it shows the opposite visible state.
3. Because the pattern is symmetric, inspect only one side from the outermost wing to the midge.
4. Count every $0 \leftrightarrow 1$ transition.
5. For each transition boundary, apply the corresponding layer-width version of the edge-parity algorithm.
6. Continue until all states are 0 and the last edge is paired.

9. Why the Method Is Useful

The transition method replaces a rapidly growing list of parity cases with one visual rule. A 5×5 has 2 symmetric states, a 7×7 has 4, a 9×9 has 8, and an 11×11 has 16. In general, with m wing orbits there are

$$2^m$$

possible symmetric binary states. These cases do not need to be memorized one by one. The solver only needs to identify the transition boundaries and use the corresponding layer-width algorithms.

The method is intended to be understandable at two levels. A young beginner can use the pictures and simply count the changes. An older student can continue to the induction and see why the same rule works for every odd-order cube.

10. Concluding Remarks

After complete reduction, odd-order cubes do not leave a genuine single-edge-flip or isolated two-edge-exchange state in the effective 3×3 stage. The parity discussed here belongs to the last-edge stage of edge pairing. The binary notation provides a simple visual language, while the underlying behavior is constrained by wing permutation within wing orbits and by the midge reference.

The 5×5 provides the base case. Going from $N \times N$ to $(N+2) \times (N+2)$ adds one new outer wing orbit, one new binary state, and at most one new transition. This gives a natural inductive proof for all odd-order cubes.

Count the transitions. Use the algorithm for each transition boundary.

Acknowledgments

The author gratefully acknowledges Lee Dearn (stoic) of the SpeedSolving.com community for his helpful explanations concerning edge parity on odd-order cubes, particularly the distinction between edge parity during edge pairing and OLL/PLL parity after reduction, and the behavior of corresponding wing pieces. The author also thanks SpeedSolving.com member bcube for emphasizing the wing-orbit and permutation viewpoint, especially the insight that apparent wing orientation is determined by permutation within a wing orbit. Their comments helped motivate the transition-based interpretation and the clearer distinction between visible binary states and the underlying physical behavior developed in this notebook.